Relatório 25 - Prática: Processamento de Linguagem Natural (NLP) (III)

Matheus Beuren Carvalho

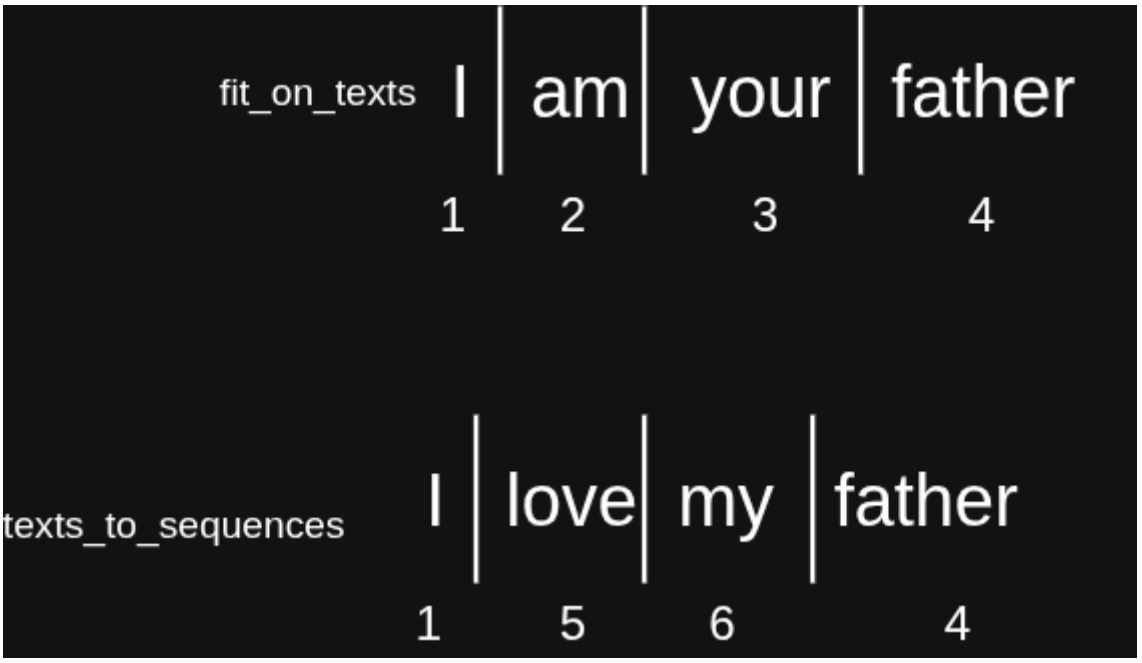
1. Introdução

O desenvolvimento deste card se resume ao aprendizado de NLP(Natural Language Processing) e como ele é aplicado a diversos conceitos como busca por palavras parecidas através da mesma classe gramatical e etc. Ao longo do relatório serão explicados os principais conceitos aprendidos durantes os dois cursos disponibilizados no card.

2. Desenvolvimento

O NLP(natural language processing), é uma subárea de inteligência artificial que faz o computador entender as palavras dos humanos, esse processo é chamado de tokenização. Esse método funciona da seguinte forma, dado uma palavra qualquer, como “I am your father” , o tokenizer vai numerar cada palavra, e no caso de vir mais uma frase, como “I am your brother” a sequência já enumerada passa a apenas repetir os números, na figura 1 podemos ver um exemplo visual de como funcionaria uma tokenização simples.

Figura 1 - Tokenização



A figura 1 mostra por meio de duas frases como a tokenização funciona, a primeira frase é separada cada palavra com um número em sequência. Após dar entrada a outra frase, percebemos que as mesmas palavras ja tokenizadas recebem o mesmo número, pois como dito, já foram tokenizadas, porém as que ainda não foram(palavras novas) serão tokenizadas seguindo a sequência. Na função do tokenizer, o 'text_to_sequences' ao aplicar com frases ja tokenizadas, ele retorna uma lista contendo os índices de cada palavra. Ao receber palavras novas, ele descarta as novas, pois não estão no seu catálogo de índices. Para resolver o problema de uma palavra fora do vocabulário, temos o parâmetro 'OOV' que ao identificar uma palavra fora do vocabulário colocando um 'Out Of Vocabulary' para assim manter a sequência dos tokens. Por mais que isso resolva, para o treino de uma rede com frases tokenizadas, é utilizado o 'padding' ou preenchimento em português, com ele é adicionado zeros a lista onde faltam índices, para que todas fiquem do mesmo tamanho, assim sendo possível realizar o treino de uma rede.

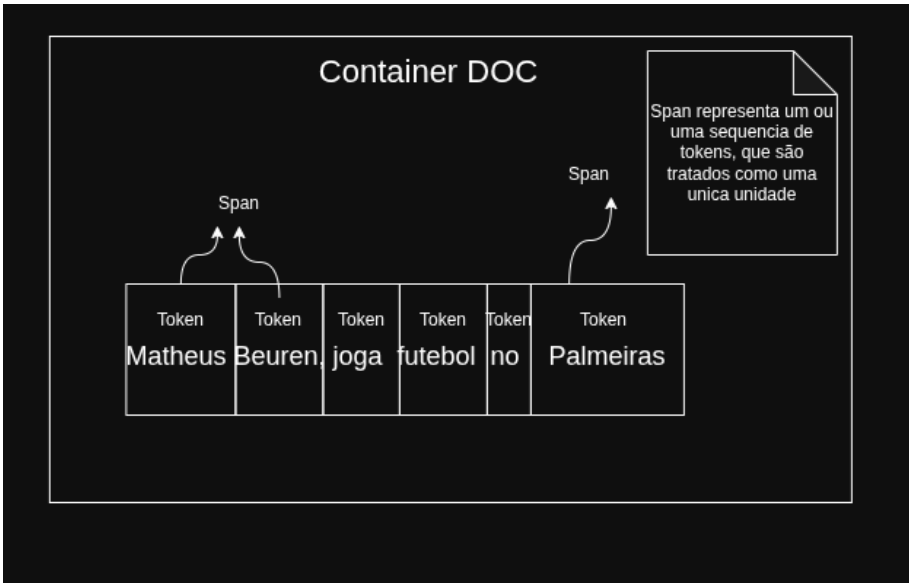
Ao longo de uma rede neural recorrente, a informação do início da frase vai se diluindo conforme a rede processa palavras maiores, então o modelo pode perder contexto importante e fazer previsões menos certas. Para resolver isso, existe o LSTM, que processa cada palavra mantém um estado para o neurônio como uma memória que é atualizada a cada palavra, decidindo o que manter e o que eliminar, permitindo que informação relevante do início da frase ainda influencie o final dela.

A área que faz uma maquina entender a linguagem dos humanos é definida como NLP, porém existe uma subárea que também é muito importante para esse aprendizado, ela se chama NLU, esta subárea se define pelo entendimento de uma frase, ou seja, o que realmente ela significa, um exemplo é a semântica de um texto.

O NLP(Natural Language Processing) se resume a ler e entender dados não estruturados e a partir deles classificar textos ou categorizá-los. Anteriormente na playlist do TensorFlow o que fizemos foi uma classificação de texto, onde foi processado e treinado uma rede neural recorrente para que a frase inserida seja classificada como tendo sarcasmo e não tendo sarcasmo.

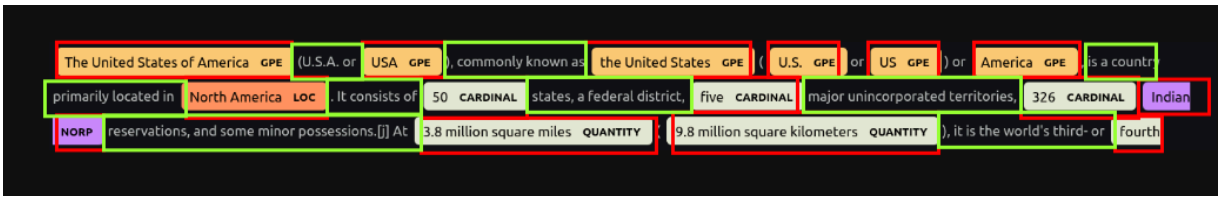
O início do curso se baseou na instalação e explicação sobre o framework spaCy. Este framework irá ajudar na construção de pipelines de NLP. Principal objeto do spaCy é o Doc, que ao passar um determinado texto guarda todos os dados processados do texto. O token é o menor objeto dentro de um Doc, e representa uma unidade ou autocontida em mais palavras(ex: "Matheus" é um token). Span é o conjunto de um ou mais tokens, seguindo o exemplo dos nomes, "Matheus Beuren Carvalho" são três tokens, que juntos estão dentro de um span.

Figura 2 - Exemplo de um doc



Ao realizar uma introdução e manipulação de alguns comandos com o spaCy, temos um exemplo visual onde podemos observar na figura 3, que, temos as entidades que estão dentro de uma span e também alguns tokens soltos. Os retângulos vermelhos representam as entidades e os retângulos verdes significa u texto sem entidade.

Figura 3 - Span e entidades



Os Word vectors, são representações numéricas de palavras em um espaço multidimensional. Como já visto em outros cards, o Embedding faz essa transformação de uma sequência de índices das palavras, e os coloca em um espaço multidimensional onde cada palavra tem sua sequência de números em um vetor.

Os pipelines apresentados são tarefas executadas em uma ordem específica, assim como suas tarefas. Nesse contexto, os entity rules são regras definidas por meio do código que o spaCy vai usar para determinar as entidades em um texto.

Matcher é um mecanismo que é atribuído ao pipeline que procura tanto semanticamente, mesma classe gramatical ou operadores, todas as frases que se assemelham ou correspondem obrigatoriamente a algum atributo. Custom Components permite criar funções python que podem ser adicionadas ao pipeline.

As práticas presentes no card, se baseiam na utilização de NLP para manipulá-las e procurar semelhanças e encontrar padrões em determinadas bases. Em uma das práticas, foi utilizado uma base de filmes onde o objetivo era encontrar as ações do personagem sem precisar ler toda a sinopse. Ao acionar os dois matchers criados para a detecção de ações de personagem ao dataset inteiro, obteve-se um resultado bom com apenas 5568 ações não encontradas de 34886 sinopses de filmes.

3. Conclusão

Concluído o relatório sobre a Natural Language Processing(NLP), foram realizadas duas práticas para demonstrar o aprendizado dos cursos aplicados a diferentes datasets. Onde foram realizados experimentos com “Entity Ruler” e “Matcher”, para achar na base de vagas na área de ti os vagas pela suas skills, e na base de filmes, achar as ações do personagem sem precisar ler toda a sinopse do filme. Os conceitos aprendidos durante as aulas e na prática consistem nas principais formas em que a máquina entende a linguagem humana e como manipulá-la para realizar buscas e semelhanças em textos.